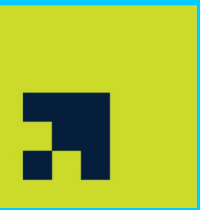


Code Style

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



Pythonic Coding

- Readability is at the heart of Python's design
 - Following the recognition that code is read much more often than it is written
- One reason Python code can be easily understood is its relatively complete set of code style guidelines
 - Collected in the two Python Enhancement Proposals PEP 20 and PEP 8, and “Pythonic” idioms
- Not “Pythonic” codes
 - Do not follow the common guidelines
 - Fail to express the intent in what is considered the most readable way



PEPs

- PEPs are Python Enhancement Proposals
 - They describe changes to Python itself, or the standards around it
 - People interested in Python's history, or in language design in general, would find them all really interesting
 - Even the ones that eventually get rejected
- There are a few PEPs that could be considered required reading
 - PEP 8: Style guide for Python code
 - Read this
 - All of it
 - Follow it
 - PEP 20: The zen of Python
 - It is a list of 19 statements that briefly explain the philosophy behind Python
 - PEP 257: Docstring conventions
 - It contains the guidelines for semantics and conventions associated with Python docstrings



PEP 8: Style Guide for Python Code

- PEP 8 gives coding conventions for the Python code
 - This style guide evolves over time as additional conventions are identified and past conventions are rendered obsolete by changes in the language itself
- Many projects have their own coding style guidelines
 - In the event of any conflicts, such project-specific guides take precedence for that project



A Foolish Consistency is the Hobgoblin of Little Minds

- Code is read much more often than it is written
 - The guidelines provided here are intended to improve the readability of code and make it consistent across the wide spectrum of Python code
- A style guide is about consistency
 - Consistency with this style guide is important
 - Consistency within a project is more important
 - Consistency within one module or function is the most important
- However, know when to be inconsistent
 - Sometimes style guide recommendations just are not applicable
 - When in doubt, use your best judgment
 - Look at other examples and decide what looks best
 - In particular: Do not break backwards compatibility just to comply with this PEP



Indentation

- Use 4 spaces per indentation level

Wrong:

Arguments on first line forbidden when not using vertical alignment.

```
foo = long_function_name(var_one, var_two,  
    var_three, var_four)
```

Further indentation required as indentation is not distinguishable.

```
def long_function_name(  
    var_one, var_two, var_three,  
    var_four):  
    print(var_one)
```

Correct:

Aligned with opening delimiter.

```
foo = long_function_name(var_one, var_two,  
    var_three, var_four)
```

Add 4 spaces (an extra level of indentation) to distinguish arguments from the rest.

```
def long_function_name(  
    var_one, var_two, var_three,  
    var_four):  
    print(var_one)
```

Hanging indents should add a level.

```
foo = long_function_name(  
    var_one, var_two,  
    var_three, var_four)
```



Tabs or Spaces?

- Spaces are the preferred indentation method
 - Tabs should be used solely to remain consistent with code that is already indented with tabs
- Python 3 disallows mixing the use of tabs and spaces for indentation



Maximum Line Length

- Limit all lines to a maximum of 79 characters
 - Limiting the required editor window width makes it possible to have several files open side by side, and works well when using code review tools that present the two versions in adjacent columns
 - Long lines can be broken over multiple lines by wrapping expressions in parentheses
 - These should be used in preference to using a backslash for line continuation
- For flowing long blocks of text with fewer structural restrictions (docstrings or comments), the line length should be limited to 72 characters



Should a Line Break Before or After a Binary Operator?

- For decades the recommended style was to break after binary operators
 - But this can hurt readability in two ways:
 - The operators tend to get scattered across different columns on the screen
 - Each operator is moved away from its operand and onto the previous line
- Mathematicians and their publishers follow the opposite convention
 - To solve this readability problem

```
# Wrong:  
# operators sit far away from their operands  
income = (gross_wages +  
          taxable_interest +  
          (dividends - qualified_dividends) -  
          ira_deduction -  
          student_loan_interest)
```

```
# Correct:  
# easy to match operators with operands  
income = (gross_wages  
          + taxable_interest  
          + (dividends - qualified_dividends)  
          - ira_deduction  
          - student_loan_interest)
```



Blank Lines

- Surround top-level function and class definitions
 - With two blank lines
- Method definitions inside a class
 - Are surrounded by a single blank line
- Extra blank lines may be used (sparingly)
 - To separate groups of related functions
 - Blank lines may be omitted between a bunch of related one-liners
- Use blank lines in functions sparingly
 - To indicate logical sections



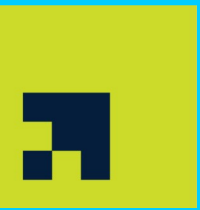
Imports

- Imports should usually be on separate lines
- Imports are always put at the top of the file
 - Just after any module comments and docstrings
 - And before module globals and constants

```
# Wrong:  
import sys, os
```

```
# Correct:  
import os  
import sys
```

```
# Correct:  
from subprocess import Popen, PIPE
```



Imports

- Imports should be grouped
 - Standard library imports
 - Related third party imports
 - Local application/library specific imports
- You should put a blank line between each group of imports

```
import os
import sys

import numpy
import pandas

import mymodule
```



Imports

- Importing a class from a class-containing module is usually okay
 - If this spelling causes local name clashes, then spell them explicitly
- Wildcard imports (from <module> import *) should be avoided
 - As they make it unclear which names are present in the namespace, confusing both readers and many automated tools

Very bad (confusing for a reader)	Better (obvious which new names are in the global namespace)	Best (immediately obvious where the attribute comes from)
<code>from modu import *</code>	<code>from modu import sqrt</code>	<code>import modu</code>
<code>x = sqrt(4)</code>	<code>x = sqrt(4)</code>	<code>x = modu.sqrt(4)</code>
Is <code>sqrt</code> part of <code>modu</code> ? Or a built-in? Or defined above?	Has <code>sqrt</code> been modified or redefined in between, or is it the one in <code>modu</code> ?	Now <code>sqrt</code> is visibly part of <code>modu</code> 's namespace.



String Quotes

- In Python, single-quoted strings and double-quoted strings are the same
 - This PEP does not make a recommendation for this
 - Pick a rule and stick to it
 - When a string contains single or double quote characters, however, use the other one to avoid backslashes in the string
 - It improves readability
- For triple-quoted strings, always use double quote characters
 - To be consistent with the docstring convention in PEP 257



Whitespace in Expressions and Statements

- Avoid extraneous whitespace in the following situations:
 - Immediately inside parentheses, brackets or braces
 - Between a trailing comma and a following close parenthesis
 - Immediately before a comma, semicolon, or colon

```
# Wrong:  
spam( ham[ 1 ], { eggs: 2 } )  
  
# Correct:  
spam(ham[1], {eggs: 2})
```

```
# Wrong:  
bar = (0, )  
  
# Correct:  
foo = (0,)
```

```
# Wrong:  
if x == 4 : x , y = y , x  
  
# Correct:  
if x == 4: x, y = y, x
```



Whitespace in Expressions and Statements

- However, in a slice the colon acts like a binary operator
 - It should have equal amounts on either side
 - Treating it as the operator with the lowest priority
 - In an extended slice, both colons must have the same amount of spacing applied
 - Exception: When a slice parameter is omitted, the space is omitted

```
# Wrong:  
ham[lower + offset:upper + offset]  
ham[1: 9], ham[1 :9], ham[1:9 :3]  
ham[lower : : upper]  
ham[ : upper]
```

```
# Correct:  
ham[1:9], ham[1:9:3], ham[:9:3], ham[1::3], ham[1:9:]  
ham[lower:upper], ham[lower:upper:], ham[lower::step]  
ham[lower+offset : upper+offset]  
ham[: upper_fn(x) : step_fn(x)], ham[:: step_fn(x)]  
ham[lower + offset : upper + offset]
```



Whitespace in Expressions and Statements

- Avoid extraneous whitespace in the following situations:
 - Immediately before the open parenthesis that starts the argument list of a function call
 - Immediately before the open parenthesis that starts an indexing or slicing
 - More than one space around an assignment (or other) operator to align it with another

```
# Wrong:  
spam (1)
```

```
# Correct:  
spam(1)
```

```
# Wrong:  
dct ['key'] = lst [index]
```

```
# Correct:  
dct['key'] = lst[index]
```

```
# Wrong:  
x           = 1  
y           = 2  
long_variable = 3
```

```
# Correct:  
x = 1  
y = 2  
long_variable = 3
```



Other Whitespace Recommendations

- Avoid trailing whitespace anywhere
 - Because it is usually invisible, it can be confusing
- Always surround the binary operators
 - With a single space on either side
- If operators with different priorities are used
 - Consider adding whitespace around the operators with the lowest priority(ies)
- Do not use spaces around the = sign
 - When used to indicate a keyword argument or
 - When used to indicate a default value for an unannotated function parameter
 - `def function(condition=True)`



Compound Statements

- Compound statements (multiple statements on the same line) are generally discouraged
 - While sometimes it is okay to put an if/for/while with a small body on the same line
 - Never do this for multi-clause statements
 - Also avoid folding such long lines

```
# Wrong:  
if foo == 'blah': do_blah_thing()  
do_one(); do_two(); do_three()
```

```
# Wrong:  
if foo == 'blah': do_blah_thing()  
for x in lst: total += x  
while t < 10: t = delay()
```

```
# Correct:  
if foo == 'blah':  
    do_blah_thing()  
do_one()  
do_two()  
do_three()
```



Comments

- Comments that contradict the code are worse than no comments
 - Always make a priority of keeping the comments up-to-date when the code changes
- Comments should be complete sentences
 - The first word should be capitalized, unless it is an identifier that begins with a lower case letter
 - Never alter the case of identifiers



Inline Comments

- An inline comment is a comment on the same line as a statement
 - Inline comments should be separated by at least two spaces from the statement
 - They should start with a # and a single space
- Inline comments are unnecessary and in fact distracting if they state the obvious
 - Use inline comments sparingly

```
x = x + 1          # Increment x
```



Documentation Strings

- Write docstrings for all public modules, functions, classes, and methods
 - Docstrings are not necessary for non-public methods, but you should have a comment that describes what the method does
 - This comment should appear after the def line
- PEP 257 describes good docstring conventions
 - Most importantly, the `"""` that ends a multiline docstring should be on a line by itself
 - For one liner docstrings, please keep the closing `"""` on the same line

```
"""Return a foobang  
  
Optional plotz says to frobnicate the bizbaz first.  
"""
```

```
"""Return an ex-parrot."""
```



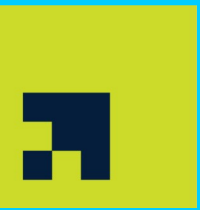
Naming Conventions

- The naming conventions of Python's library are a bit of a mess
 - So, we will never get this completely consistent
 - Nevertheless, here are the currently recommended naming standards
- Never use the characters 'l' (lowercase letter el), 'O' (uppercase letter oh), or 'I' (uppercase letter eye) as single character variable names
 - In some fonts, these characters are indistinguishable from the numerals one and zero
 - When tempted to use 'l', use 'L' instead



Naming Conventions

- Function, method, and variable names
 - Should be lowercase, with words separated by underscores as necessary to improve readability
 - E.g., my_function
- Constants
 - Are usually defined on a module level and written in all capital letters with underscores separating words
 - Examples include MAX_OVERFLOW and TOTAL
- Class names
 - Should normally use the CapWords convention
- Use one leading underscore
 - Only for non-public methods and instance variables
- Python packages
 - Should also have short, all-lowercase names, although the use of underscores is discouraged
- mixedCase
 - Is allowed only in contexts where that is already the prevailing style (e.g. threading.py), to retain backwards compatibility



Naming Conventions

- Always use self
 - For the first argument to instance methods
- Always use cls
 - For the first argument to class methods
- If a function argument's name clashes with a reserved keyword
 - It is generally better to append a single trailing underscore rather than use an abbreviation or spelling corruption
 - Thus, `class_` is better than `clss`
 - Perhaps better is to avoid such clashes by using a synonym



Explicit is Better Than Implicit

- In the good code, x and y are explicitly received from the caller
 - And an explicit dictionary is returned
- Another developer should be able to read the first and last lines of your function and understand what it does

Bad

```
def make_dict(*args):  
    x, y = args  
    return dict(**locals())
```

Good

```
def make_dict(x, y):  
    return {'x': x, 'y': y}
```